

7. Technical Debt Analysis

Objective: Establish the prerequisites for an Agentic Harness & Marketplace initiative — code repository health, third-party tool usage, AI tool usage, database usage, and development environment readiness.

Date: July 16, 2026 | **Scope:** `target/` — React 18 SPAs (Create React App 5): `social-media-react` (JavaScript/JSX, Redux, Socket.io) and `workbench-demo` (TypeScript/TSX); AI orchestration configs (`.cursor/` , `.kiro/` , `.claude/`); no server-side or database source in workspace

Executive Summary

EXECUTIVE SUMMARY

TARGET_WORKSPACE hosts two independent React 18 frontends plus mature AI-orchestration scaffolding (MCP configs, pipeline history, knowledge base), but **lacks the baseline engineering gates an agentic harness requires**. The three most severe gaps are: **(1) zero project-level CI/CD workflows**, so no automated lint/test/build gate exists to accept agent-authored changes; **(2) live API tokens committed in nine `mcp.json` files** under `.cursor/`, `.claude/`, and `.kiro/settings/`, creating a secrets-exposure and clean-checkout risk for automation; and **(3) no `.env.example`, Dockerfile, or enforced lint/format pipeline**, leaving onboarding and environment parity manual. Third-party dependencies are mostly wired (Axios, Socket.io, Redux), but `jwt-decode`, `dotenv`, and seven `workbox-*` packages are declared yet unused. Database schema and migrations are entirely external to this workspace. **Agentic-harness readiness today is High Risk**, driven by repository hygiene (D1), development environment (D5), and credential management (D6).

Partial

TOP-LEVEL .GITIGNORE PRESENT

0

CI/CD WORKFLOWS FOUND

10 / 15

THIRD-PARTY PACKAGES DECLARED / WIRED

No

.ENV.EXAMPLE PRESENT

OVERALL CODEBASE RATING — TECHNICAL DEBT & AGENTIC READINESS

High Risk

Driven by High-Risk gaps in Code Repository Health (D1), Development Environment (D5), and Credential Hygiene (D6) — no CI gate and committed secrets block safe agent automation.

Readiness Benchmark Ratings

#	Dimension	GOOD	MODERATE	HIGH RISK	Measured	Rating
---	-----------	------	----------	-----------	----------	--------

D1	Code Repository Health	all checks pass	1–2 gaps	3+ gaps / no CI	No top-level <code>.gitignore</code> ; <code>workbench-demo</code> lacks <code>.gitignore</code> ; 0 CI workflows; lock files present in both apps; no PR template/CODEOWNERS	HIGH RISK
D2	Third-Party Tool Usage	mostly wired & current	some unused/unwired	many unused or unmaintained	10 of 15 security/infrastructure packages wired; <code>jwt-decode</code> , <code>dotenv</code> , 7 <code>workbox-*</code> packages declared but unused	MODERATE
D3	AI Tool / Agentic Readiness	ready	partial	not ready	Rich <code>.cursor</code> / <code>.kiro</code> / <code>.claude</code> configs and knowledge base; 11 enumerable domain services; but no CI gate, dual-app paradigm split, stale default test	MODERATE
D4	Database Usage	sound	some gaps	no constraints / shared flat schema	No schema, migrations, or seed scripts in workspace; persistence assumed external (MongoDB per README)	MODERATE
D5	Development Environment	reproducible	partial	manual / fragile	No <code>.env.example</code> , no containerization, ESLint configured but not enforced; README documents nonexistent <code>npm run lint</code>	HIGH RISK
D6	Credential / Secrets Hygiene (additional)	tokens externalized & gitignored	some config files hold secrets	live tokens in tracked configs	9 <code>mcp.json</code> files contain <code>ATLASSIAN_API_TOKEN</code> and <code>GITHUB_PERSONAL_ACCESS_TOKEN</code> literals	HIGH RISK

7.1 Code Repository

Check	Files Inspected	Finding	Consequence	Next Step
Top-level <code>.gitignore</code>	<code>target/</code> (root)	Missing — no <code>target/.gitignore</code> exists	Workspace-level artifacts (<code>.env</code> , orchestration outputs, local <code>node_modules</code>) at <code>target/</code> root are not excluded by any ignore rules	Add <code>target/.gitignore</code> , <code>node_modules</code> , <code>agent-runs/</code> , ar caches
App-level <code>.gitignore</code>	<code>target/social-media-react/.gitignore:1-25</code> , <code>target/workbench-demo/</code>	<code>social-media-react</code> covers <code>/node_modules</code> , <code>/build</code> , <code>.env*</code> variants; <code>workbench-demo</code> has no <code>.gitignore</code>	<code>workbench-demo/node_modules</code> (324 MB on disk) and <code>workbench-demo/coverage/</code> (192 KB) can be accidentally committed without an app-level ignore file	Copy CRA-standard <code>target/workben</code>

CI/CD presence	<code>target/**/*.github/workflows/</code> (excluding <code>node_modules</code>)	0 project workflows found — only third-party workflows inside <code>node_modules</code>	Merges to <code>main</code> are not validated by automated lint, test, or build; agent-authored PRs have no verification gate	Add <code>.github/workflows</code> at repo root running <code>run build</code>
Branch protection signals	<code>target/**/*.CODEOWNERS</code> , <code>target/**/*.PULL_REQUEST_TEMPLATE*</code>	Not found locally	No visible CODEOWNERS or PR template to enforce human review alongside CI	Add <code>.github/CODEOWNERS</code> and <code>.github/pull_request_template.md</code>
Lock files	<code>target/social-media-react/package-lock.json</code> , <code>target/workbench-demo/package-lock.json</code>	Both present and committed (1.2 MB and 662 KB respectively)	Reproducible <code>npm ci</code> installs are possible for both apps	Keep lock files updated on changes; add CI <code>run build</code>

7.2 Third-Party Tools Usage

Package	Declared	Actually Wired?	Debt Note
<code>axios</code>	Y (<code>social-media-react/package.json:15</code> , <code>workbench-demo/package.json:6</code>)	Y — <code>httpService.js:1-39</code> , <code>authService.ts:1-9</code>	Core HTTP client; hard-coded <code>localhost:3030</code> in <code>social-media-react</code> instead of env var
<code>socket.io-client</code>	Y (<code>social-media-react/package.json:28</code>)	Y — <code>socket.service.js:1-33</code>	Wired; connects to <code>//localhost:3030</code> with no env override
<code>redux</code> / <code>react-redux</code> / <code>redux-thunk</code>	Y (<code>social-media-react/package.json:21-23,25</code>)	Y — <code>store/index.js</code> , <code>store/actions/*.js</code>	Fully wired state layer
<code>react-router-dom</code>	Y (v5 in <code>social-media-react</code> , v6 in <code>workbench-demo</code>)	Y — route pages in both apps	Version split (v5 vs v6) increases agent refactor surface
<code>google-map-react</code>	Y (<code>social-media-react/package.json:17</code>)	Y — <code>Map.jsx:2,24-30</code>	Wired for map feature
<code>@strg/react-snip</code>	Y (<code>social-media-react/package.json:11</code>)	Y — <code>MsgPreview.jsx:2</code>	Wired in messaging UI
<code>jwt-decode</code>	Y (<code>social-media-react/package.json:18</code>)	N — no imports in <code>src/</code>	Declared for auth but unused; auth uses <code>localStorage</code> via <code>userService.getLoggedInUser()</code>
<code>dotenv</code>	Y (<code>social-media-react/package.json:16</code>)	N — no imports in <code>src/</code>	Redundant with CRA's built-in env loading
<code>workbox-core</code>	Y	Y — <code>service-worker.js:10</code>	Used in PWA service worker
<code>workbox-expiration</code>	Y	Y — <code>service-worker.js:11</code>	Used

<code>workbox-precaching</code>	Y	Y — <code>service-worker.js:12</code>	Used
<code>workbox-routing</code>	Y	Y — <code>service-worker.js:13</code>	Used
<code>workbox-strategies</code>	Y	Y — <code>service-worker.js:14</code>	Used
<code>workbox-background-sync</code>	Y	N	Declared but never imported
<code>workbox-broadcast-update</code>	Y	N	Declared but never imported
<code>workbox-cacheable-response</code>	Y	N	Declared but never imported
<code>workbox-google-analytics</code>	Y	N	Declared but never imported
<code>workbox-navigation-preload</code>	Y	N	Declared but never imported
<code>workbox-range-requests</code>	Y	N	Declared but never imported
<code>workbox-streams</code>	Y	N	Declared but never imported

No files matched `target/social-media-react/package.json` containing `workbox-(background-sync|broadcast-update|cacheable-response|google-analytics|navigation-preload|range-requests|streams)`.

7.3 AI Tool Usage & Agentic Readiness

Existing AI tooling (mature orchestration layer):

- `target/.cursor/mcp.json:1-99` — Cursor MCP server definitions for Jira, GitHub, observability
- `target/.kiro/settings/mcp.json` — Kiro-equivalent MCP bundle with Jira/GitHub integration
- `target/.claude/mcp.json` — Claude MCP mirror config
- `target/.kiro/context/knowledge-base/` — `error-patterns.md`, `resolutions.md`, `root-causes.md` (institutional memory for agents)
- `target/.kiro/context/pipeline-history/index.md` — MAD-5, MAD-81, MAD-102 incident pipeline records
- Nested copies under `target/workbench-demo/.cursor/`, `target/ServiceCategoryInputFiles/.cursor/` (duplicated MCP configs)

Code generators / scaffolding: No dedicated codegen scripts (e.g., Plop, Hygen) found. Agent definitions live in the parent orchestration catalog (`multi-agent-web-ui/.cursor/agents/`), not in `TARGET_WORKSPACE` application source.

Enumerable units of work:

Unit Type	Count	Location	Agent Refactor Potential
Domain services	11	<code>target/social-media-react/src/services/**/*.js</code>	High — uniform CRUD-via- <code>httpService</code> pattern
Redux action modules	4	<code>target/social-media-react/src/store/actions/*.js</code> (544 LOC total)	High — thunk-per-domain pattern
Route pages (social-media)	13	<code>target/social-media-react/src/pages/*.jsx</code>	Moderate — mixed direct-service vs dispatch calls
TS pages (workbench-demo)	4	<code>target/workbench-demo/src/pages/*.tsx</code>	High — small, typed, test-covered
View-layer service bypasses	11	Files calling <code>userService</code> directly (see architecture report H2)	High — enumerable migration to action layer

Assessment: AI orchestration infrastructure is **ahead of application engineering maturity** — agents have MCP access, knowledge base, and pipeline history, but the application repos they target lack CI verification, consistent patterns across the two SPAs, and a trustworthy test baseline (`social-media-react/src/App.test.js:4-7` still asserts `/learn react/i`, which does not exist in the current `App.js` routing shell). An agent could enumerate service-layer refactors systematically, but **cannot safely auto-merge** without a CI gate.

No files matched `target/social-media-react/src/**/*.{jsx,js}` containing `userService\.(getById|getUsers|login|update)`.

7.4 Database Usage

Check	Files Inspected	Finding	Consequence	Next Step
Schema design	<code>target/**/*.{sql,prisma,schema*}</code> — 0 files	No database schema, ORM models, or migration files exist in <code>TARGET_WORKSPACE</code>	Foreign keys, indexes, and normalization cannot be verified locally; all persistence is assumed in the external backend (Social Network Backend per <code>social-media-react/README.md:9</code>)	Clone/link backend repo into workspace or document API contract schema for agent reference
Migration hygiene	<code>target/**/*.migrations/**</code> — not found	No migration history visible	Schema changes are opaque to frontend agents; destructive migrations cannot be assessed	Add backend as git submodule or publish OpenAPI/JSON Schema for API entities
Data ownership	N/A locally	Frontend uses flat REST endpoints (<code>user</code> , <code>post</code> , <code>chat</code>) with no domain-scoped schema	Shared flat API surface mirrors monolithic backend coupling	Define bounded-context API contracts when extracting services

Seed/sample data hygiene	<code>target/**/*.seed*</code> , <code>target/**/*.fixtures/**</code> — not found	No seed scripts in workspace	Agents cannot spin up reproducible data scenarios locally	Add <code>fixtures/</code> or MSW mock handlers for offline agent testing
--------------------------	--	------------------------------	---	---

Note: Absence of local DB artifacts is expected for a frontend-only checkout, but it is a **moderate modernization risk** because agents working on data-validation or migration tasks have no local source of truth.

7.5 Development Environment

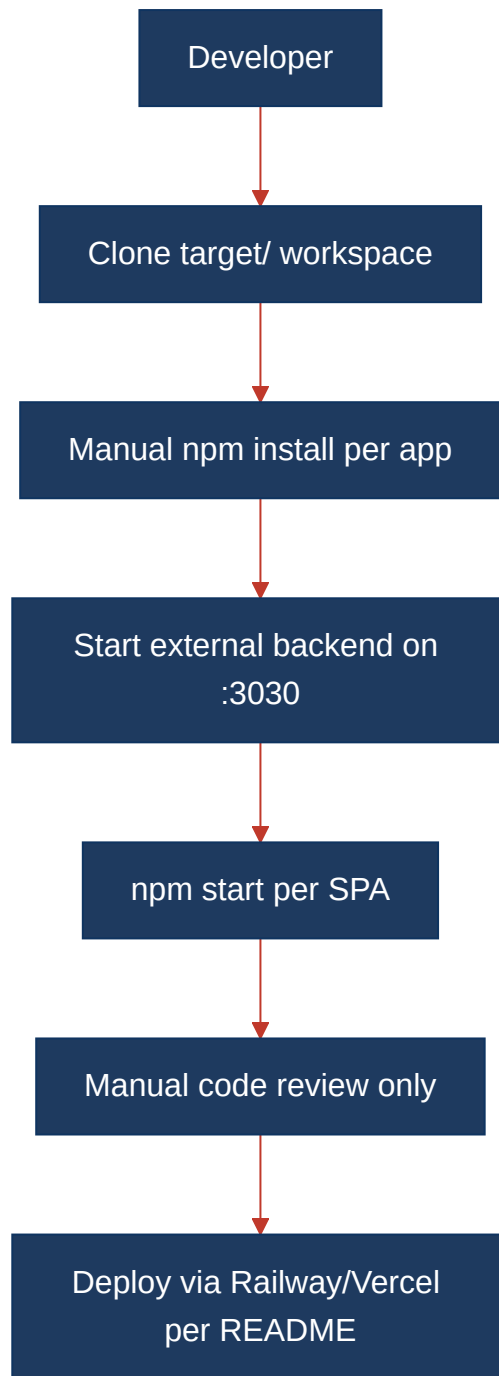
Check	Files Inspected	Finding	Consequence	Next
<code>.env.example</code>	<code>target/**/*.env.example</code> — 0 files	Neither app nor workspace root provides an example env file	<code>workbench-demo/src/services/authService.ts:6</code> requires <code>REACT_APP_API_BASE_URL</code> with no documented default; new contributors and CI runners cannot discover required vars	Add <code>REA</code> and docu
OS portability	<code>social-media-react/README.md:21-49</code> , both <code>package.json</code> scripts	Standard <code>npm install / npm start</code> — OS-portable	Works on Linux/macOS/Windows via Node.js	Docu <code>pac</code>
Containerization	<code>target/**/*.Dockerfile</code> , <code>target/**/*.docker-compose*.yaml</code> , <code>target/**/*.devcontainer/**</code> — 0 files	No container or devcontainer config	Environment drift between contributors; agents cannot assume reproducible container baseline	Add back
Code style enforcement	<code>social-media-react/package.json:49-54</code> (eslintConfig), <code>workbench-demo/package.json</code> (no eslintConfig), no <code>.husky/</code> or <code>.pre-commit-config.yaml</code>	ESLint configured in social-media-react via CRA defaults but not enforced in CI or pre-commit hooks; workbench-demo has no ESLint config block; README claims <code>npm run lint</code> (<code>README.md:56</code>) but no lint script exists in <code>package.json:43-48</code>	Style drift is immediate; documented lint command fails; agents cannot rely on lint gate	Add wire i
Backend dependency	<code>httpService.js:3-4</code> , <code>socket.service.js:6</code>	Hard-coded <code>localhost:3030</code> for dev API and WebSocket	Agents and contributors must manually start external backend; no health-check or startup script	Add comr

7.6 Prerequisites for Agentic Harness & Marketplace Readiness

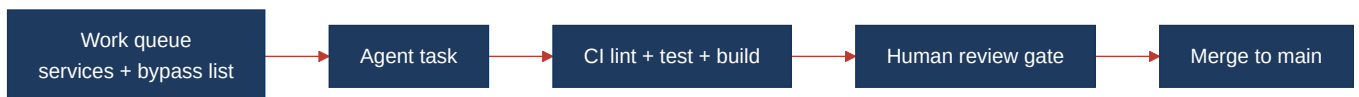
Prerequisite	Current State	Gap
Enumerable work queue (clear, listable units of refactor work)	11 domain services + 11 view-layer bypass files + 4 Redux action modules provide listable targets	Partial — queue exists but spans two apps with different paradigms (JS/v5 vs TS/v6)
Isolated, verifiable units of work	<code>workbench-demo</code> has isolated <code>authService.ts</code> with co-located tests; <code>social-media-react</code> services are isolated but views mix concerns	Partial — social-media-react lacks test coverage beyond stale CRA default
CI gate to accept agent-authored output	0 CI workflows in project scope	CRITICAL — no automated verification path for agent PRs
Repo hygiene for automation (clean checkout, no secrets)	Lock files present; 9 <code>mcp.json</code> files contain live API tokens ; <code>workbench-demo</code> missing <code>.gitignore</code>	CRITICAL — secrets exposure and missing ignore rules
Marketplace packaging readiness	AI orchestration configs exist but application repos are not independently publishable packages	HIGH — no monorepo tooling, no per-app CI, no version strategy

7.7 Diagrams

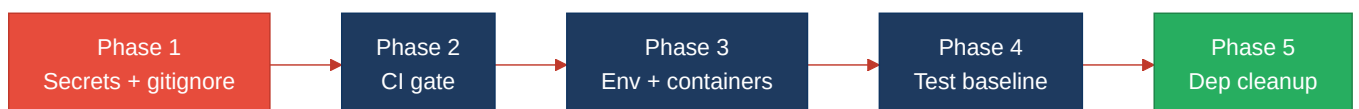
Current dev / delivery flow



Agentic harness readiness target



Improvement roadmap



7.8 Actions Required

Gap	Action	Rating	Priority
No CI/CD workflows — agent changes unverified	Add <code>.github/workflows/ci.yml</code> running <code>npm ci</code> , <code>npm test -- --watchAll=false</code> , and <code>npm run build</code> for <code>social-media-react</code> and <code>workbench-demo</code> on every PR	HIGH RISK	CRITICAL
Live API tokens in 9 tracked <code>mcp.json</code> files	Rotate <code>ATLASSIAN_API_TOKEN</code> and <code>GITHUB_PERSONAL_ACCESS_TOKEN</code> ; remove literals from all <code>mcp.json</code> files; load from environment; add <code>**/mcp.json</code> env blocks to <code>.gitignore</code> or use <code>.env</code> references	HIGH RISK	CRITICAL
Missing top-level and workbench-demo <code>.gitignore</code>	Add <code>target/.gitignore</code> and <code>target/workbench-demo/.gitignore</code> covering <code>node_modules/</code> , <code>coverage/</code> , <code>.env</code> , and build output	HIGH RISK	HIGH
No <code>.env.example</code> — undocumented <code>REACT_APP_API_BASE_URL</code>	Create <code>.env.example</code> in both apps documenting required and optional environment variables	HIGH RISK	HIGH
ESLint configured but not enforced; README lint script missing	Add <code>lint</code> npm script to both apps; wire <code>npm run lint</code> into CI workflow; fix or remove stale <code>npm run lint</code> reference in <code>social-media-react/README.md:56</code>	HIGH RISK	MEDIUM
No containerization path	Add <code>docker-compose.yml</code> at workspace root with frontend services and documented backend dependency on port 3030	HIGH RISK	MEDIUM
Unused dependencies (<code>jwt-decode</code> , <code>dotenv</code> , 7 <code>workbox-*</code>)	Remove unwired packages from <code>social-media-react/package.json</code> or integrate them (e.g., <code>jwt-decode</code> in <code>PrivateRoute.jsx</code> auth check)	MODERATE	LOW
No local database schema/migrations	Link backend repo or add API schema artifacts (OpenAPI/JSON Schema) for agent data-validation tasks	MODERATE	MEDIUM
Stale default test (<code>App.test.js</code> expects "learn react")	Rewrite <code>social-media-react/src/App.test.js</code> to match current routing shell or add MSW-backed integration tests	MODERATE	MEDIUM
Dual-app paradigm split (JS/Router v5 vs TS/Router v6)	Document app boundaries; prioritize TypeScript migration roadmap for <code>social-media-react</code> as enumerable agent work queue	MODERATE	MEDIUM

7.9 Expected Outcomes

- Clean, reproducible checkout with `.gitignore`, `.env.example`, and no committed secrets — safe for agent automation clones
- CI gate (lint + test + build) trusts agent-authored PRs before human review
- Removed unused dependencies reduce install surface and supply-chain risk
- Containerized dev path eliminates manual backend-port tribal knowledge

- Enumerable service-layer and view-bypass refactors become agent-ready work queue items with verifiable test coverage

No files matched `target/**/*.mcp.json` containing `ATLASSIAN_API_TOKEN|GITHUB_PERSONAL_ACCESS_TOKEN` .